VoaÁguia

Documentação técnica da Sprint 2

Challenge Grupo Águia Branca • FIAP • 2026

Backend Java, aplicativo Android integrado e gestão da inovação com contexto estratégico.

Equipe

Davi Simplicio Nunes Coelho — RM 564403

Guilherme Vinicius Gonçalves da Silva — RM 564401

Gustavo Okamoto Ventura — RM 564945

Mirella Ayumi Miyakawa — RM 564426

Esta documentação descreve o código da Sprint 2. A integração de IA requer chave própria no servidor e validação com o provedor. A persistência utiliza Spring Data MongoDB; a equivalência com a menção a JPA/Hibernate no enunciado precisa ser confirmada pela equipe com o professor.

Arquitetura e decisões

Aplicativo nativo

Kotlin e Jetpack Compose preservam as telas da Sprint 1. ApiRepository usa OkHttp e Gson para consumir o backend. O JWT permanece apenas em memória; o app trata expiração retornando ao login.

API e segurança

Spring Boot organiza controllers, services, documentos e repositories. Spring Security verifica JWT e permissões por método. BCrypt protege senhas. Logout revoga o identificador do token no MongoDB com expiração TTL.

Dados NoSQL

MongoDB armazena usuários, estratégias, ideias, projetos e tokens revogados. Estratégias incorporam versões históricas. Referências strategyId e ideaId conectam as etapas do processo. Índices únicos evitam e-mail duplicado e mais de um projeto por ideia.

Integração externa

O serviço AiService chama generateContent da API Gemini por HTTPS. A chave existe somente no ambiente do backend. Gestores podem pedir análise contextual e sugestões; líderes podem solicitar insights dos indicadores.

Execução

O aplicativo chama /api por REST/JSON; a API acessa o MongoDB e o Gemini. Docker Compose pode iniciar banco e servidor. Também é possível usar Java 17 e MongoDB local. Consulte os READMEs para comandos e configuração.

Regras e critérios

OPERADOR: cadastro público, consulta de estratégias, CRUD de ideias próprias enquanto pendentes e acompanhamento da decisão.

GESTOR: avaliação e priorização de ideias, criação de projetos a partir de aprovação, atualização de progresso e resultados, exclusão lógica de projetos e auxílio da IA.

LIDER: CRUD de estratégias, consulta dos projetos, dashboard filtrado e insights. A API bloqueia mudanças de projeto feitas por líderes.

Uma estratégia vigente é obrigatória no cadastro da ideia e do projeto. O projeto mantém a estratégia original da ideia. O servidor determina autoria e identificadores.

Lucro = retorno realizado - investimento. ROI = lucro / investimento × 100. Com investimento zero, ROI é nulo. Redução de custos é separada do retorno; produtividade é média simples.

Histórico: alterações e desativação da estratégia acrescentam uma revisão no mesmo documento, com data, categoria, campanha, conteúdo e responsável.

Validação: testes de integração executam chamadas HTTP contra uma instância real do MongoDB em base isolada. O teste de ausência da chave de IA exige resposta explícita de indisponibilidade; não comprova uma chamada real ao Gemini.

Contrato REST - VoaÁguia Sprint 2

Base: `/api`. JSON UTF-8. Todas as rotas, exceto login e cadastro, exigem `Authorization: Bearer <token>`. Datas usam ISO-8601 UTC, por exemplo `2026-09-20T12:00:00Z`. Valores monetários são números em reais. O contrato OpenAPI exportado detalha os schemas; com a API em execução, consulte `/v3/api-docs` e `/swagger-ui/index.html`.

Autenticação

Método / rota	Payload	Resposta
POST /auth/login	`{ "email":"gestor@voaaguia.local", "password":"sua-senha" }`	200 `{token, expiresAt, user}`
POST /auth/register	`{ "email":"nome@exemplo.com", "password":"senha-com-8-ou-mais", "name":"Nome", "department":"Operação" }`	201 `{token, expiresAt, user}`; role sempre OPERADOR
GET /auth/me	Sem corpo	200 usuário com id, name, email, role, department e pontuação; sem hash
POST /auth/logout	`{}`	204 sem corpo; JWT revogado

Estratégias

Todos podem consultar; somente LIDER pode alterar.

Método / rota	Payload	Resposta
GET /strategies	Sem corpo	200 lista de estratégias ativas
GET /strategies/{id}	Sem corpo	200 estratégia, inclusive desativada
GET /strategies/{id}/history	Sem corpo	200 lista `{id,date,category,campaign,title,description,isActive,changedBy}`
POST /strategies	StrategyInput abaixo	201 estratégia criada
PUT /strategies/{id}	StrategyInput abaixo	200 estratégia atualizada; nova revisão
DELETE /strategies/{id}	Sem corpo	204; desativação e registro histórico

```
{"title":"Eficiência nas rotas","description":"Reduzir desperdícios de combustível","category":"Logística","campaign":"Eficiência 2026","isActive":true}
```

Resposta de estratégia: campos do payload + `id`, `authorId`, `authorName`, `createdAt`, `updatedAt`, `version`. IDs e autoria são atribuídos pelo servidor. O histórico só é exposto pela rota específica.

Ideias

Operadores só veem as próprias ideias. Gestores e líderes podem consultá-las. Somente OPERADOR cria/edita/exclui; edição e exclusão exigem status PENDENTE. Somente GESTOR avalia.

Método / rota	Payload	Resposta
GET /ideas	Sem corpo	200 lista autorizada
GET /ideas/{id}	Sem corpo	200 ideia autorizada
POST /ideas	IdeaInput abaixo	201 ideia criada
PUT /ideas/{id}	IdeaInput abaixo	200 ideia atualizada
DELETE /ideas/{id}	Sem corpo	204; exclusão lógica
PATCH /ideas/{id}/review	ReviewInput abaixo	200 ideia com decisão
POST /ideas/{id}/vote	`{}`	200 ideia com votes; um voto por pessoa

IdeaInput:

```
{"title":"Rotas mais eficientes","description":"Agrupar entregas por
proximidade","category":"Logística","strategyId":"id-da-estrategia"}
```

ReviewInput:

```
{"status":"APROVADA","priority":"ALTA","feedback":"Aderente à campanha e viável para
piloto"}
```

Status de avaliação: EM_ANALISE, APROVADA, REJEITADA. Prioridade: BAIXA, MEDIA, ALTA, CRITICA. EM_PROJETO é derivado do vínculo persistido, não enviado na avaliação.

Resposta de ideia: payload + `id`, `authorId`, `authorName`, `department`, `status`, `priority`, `feedback`, `votes`, `createdAt`, `updatedAt`, `version`. Votantes e campos internos não são expostos.

Projetos

GESTOR faz CRUD; LIDER consulta. OPERADOR não acessa esses endpoints. Criação exige ideia aprovada e estratégia vigente igual à da ideia. Vínculos não podem ser alterados em PUT.

Método / rota	Payload	Resposta
GET /projects	Sem corpo	200 lista não excluída
GET /projects/{id}	Sem corpo	200 projeto
POST /projects	ProjectInput abaixo	201 projeto criado
PUT /projects/{id}	ProjectInput abaixo (objeto completo)	200 projeto atualizado
DELETE /projects/{id}	Sem corpo	204; exclusão lógica

{"title":"Piloto de rotas","description":"Validar o agrupamento por região","ideaId":"id-da-ideia","strategyId":"id-da-estrategia","status":"EM_ANDAMENTO","stage":"PILOTO","startDate":"2026-09-20T00:00:00Z","expectedEndDate":"2026-12-20T00:00:00Z","actualEndDate":null,"investment":1000,"expectedReturn":1800,"actualReturn":1500,"costReduction":200,"productivityGain":10,"progressPercent":50,"notes":"Primeiro resultado"}

Status: PLANEJAMENTO, EM_ANDAMENTO, CONCLUIDO, CANCELADO. Etapa: IDEACAO, VALIDACAO, DESENVOLVIMENTO, PILOTO, IMPLANTACAO, OPERACAO. Progresso e produtividade entre 0 e 100; valores financeiros não negativos; prazo não anterior ao início. CONCLUIDO exige progresso 100 e actualEndDate.

Resposta de projeto: payload + `id`, `managerId`, `managerName`, `createdAt`, `updatedAt`, `version`. O servidor define autoria e identificador.

Relatórios e ranking

Método / rota	Perfil	Payload / resposta
GET /reports/dashboard?strategyId={id}&projectId={id}	LIDER	Sem corpo; ambos os filtros opcionais. 200 resumo abaixo
GET /ranking	Todos	Sem corpo; 200 até 10 usuários `{id,name,department,points,ideasCount,approvedIdeasCount}`

Resumo: `{totalInvestment,totalReturn,profit,roi,roiAvailable,totalCostReduction,avgProductivityGain,activeProjects,completedProjects,overdueProjects,totalIdeas,approvedIdeas,projects}`. ROI pode ser nulo quando o investimento é zero. Projetos excluídos não entram nos cálculos. O ranking atribui 10 pontos por ideia não excluída e 50 por ideia aprovada ao menos uma vez.

IA via Gemini

POST com {} em todas as rotas. Resposta 200: `{ "text":"resposta real do modelo", "model":"modelo configurado", "provider":"Google Gemini" }`.

Rota	Perfil	Função
/ai/ideas/{id}/analysis	GESTOR	Pontuação e prioridade sugerida com contexto estratégico
/ai/ideas/{id}/project-suggestion	GESTOR	Objetivo, etapas, prazo e KPIs sugeridos
/ai/ideas/{id}/feedback?approving=true	GESTOR	Sugestão de feedback para aprovação ou rejeição
/ai/dashboard/insights?strategyId={id}&projectId={id}	LIDER	Insights a partir dos dados reais resumidos; filtros opcionais

Erros

400: campo inválido/JSON malformado. 401: credenciais ou JWT inválidos, expirados/revogados. 403: perfil não autorizado. 404: registro ausente ou ideia de outro operador. 409: conflito de regra, duplicidade ou alteração concorrente. 502: falha da IA externa. 503: IA sem chave configurada. Erros de negócio usam `{ "message":"descrição" }`; validações incluem `errors`. As respostas de segurança 401/403 também podem ser sem corpo.

Roteiro demonstrativo narrado - até 5 minutos

Preparação

Iniciar MongoDB e backend, instalar o APK da Sprint 2 e configurar uma chave Gemini válida no servidor. Usar as três contas do seed. Criar uma campanha de exemplo real pelo app. Gravar a tela com microfone, sem expor senhas, JWT, arquivo `.env` ou a chave da IA. Se a IA não tiver sido validada, não apresentar como concluída.

0:00-0:30 - Contexto e arquitetura

“Somos a equipe VoaÁguia. Na primeira sprint, construímos o aplicativo nativo. Nesta etapa, implementamos uma API Java com Spring Boot e MongoDB. O aplicativo agora consome os endpoints REST, e o servidor controla as permissões de operadores, gestores e líderes.”

Mostrar o diagrama da apresentação. Explicar que o cadastro público não permite escolher perfis privilegiados.

0:30-1:10 - Estratégia e histórico

Entrar como líder. Criar orientação “Eficiência nas rotas”, categoria “Logística”, campanha “Eficiência 2026”. Editar uma orientação e mostrar o histórico.

“A liderança define o direcionamento. Guardamos data, categoria, campanha e versões da orientação, preservando o contexto das decisões.”

1:10-2:00 - Operador

Entrar como operador e cadastrar ideia vinculada à estratégia. Mostrar edição de uma ideia ainda pendente.

“O colaborador relaciona sua proposta a uma estratégia vigente. Ele acompanha apenas suas próprias ideias, e essa regra também existe na API. Um cliente modificado não consegue consultar a ideia de outro operador.”

2:00-3:05 - Gestor e IA

Entrar como gestor. Selecionar a ideia, solicitar análise de IA, comentar uma sugestão e aprovar com prioridade e feedback.

“A IA recebe a ideia e a estratégia para sugerir pontuação de impacto, viabilidade e alinhamento. A resposta vem da API Gemini, chamada pelo backend. A chave não está no APK, e a aprovação continua sendo uma decisão humana.”

Não esperar a geração inteira em silêncio; explicar os critérios enquanto a resposta carrega. Respeitar o resultado real, sem simular uma resposta.

3:05-4:00 - Projeto e resultados

Criar projeto vinculado à ideia aprovada, indicar investimento e prazo. Atualizar etapa/progresso e resultados.

“Uma ideia aprovada dá origem a um projeto. O vínculo com a estratégia é preservado, e o gestor registra o progresso e os resultados realizados. Um índice único impede projetos duplicados para a mesma ideia.”

4:00-4:40 - Liderança e dashboard

Entrar como líder. Mostrar investimento, retorno, lucro, ROI e filtros.

“O backend consolida os indicadores por projeto, estratégia ou visão geral. O ROI usa o retorno realizado, e a redução de custos aparece separadamente para evitar dupla contagem. Sem investimento, o ROI é exibido como não aplicável.”

4:40-5:00 - Evidência técnica e fechamento

Mostrar brevemente os testes de integração e Swagger.

“Validamos o fluxo com HTTP e MongoDB reais e testamos restrições de acesso, isolamento entre operadores e revogação de sessão. A entrega inclui os dois códigos-fonte, o APK e a documentação dos endpoints.”